Fakultät für Informatik und Mathematik
Dr. Armin Größlinger

Winter semester 2026/2027

Exercises for Lecture: Virtual Machines and Runtime Systems

Exercise Sheet 2 (Signals, Threads)

Problem 1 (Signals)

The program `printvar.c` (see Stud.IP) increments a variable (`counter`) every second by 1. Compile the program using `gcc -Wall -std=c99 -o printvar printvar.c`.

(a) Extend the program such that when an interrupt signal (`SIGINT`) occurs, e.g., when Ctrl+C is pressed on the terminal, the current value of the counter is printed.

(b) The `while`-loop and the signal handler both access the variable `counter`. This is not safe in general because there can be race conditions. Protect the variable in the loop by deactivating interruptions temporarily. Use `sigprocmask` (see man pages `man sigprocmask` and `man sigsetops`).

(c) Extend the signal handlers such that the program terminates when the counter has the same value in two consecutive calls to the signal handler (i.e., quickly pressing Ctrl+C twice is supposed to terminate the program). What do you observe when you press Ctrl+C several times?

(d) The reason that pressing Ctrl+C twice does not terminate the program most of the time lies in the semantics of the function `nanosleep`. Read the man page (`man nanosleep`, particularly the section on `EINTR`) and fix the problem.

Problem 2 (Reentrancy and Signals)

(a) Give examples for functions that are thread-safe but not reentrant.

(b) `errno` is a global variable. What does this imply for signal handlers using functions from the C runtime system?

Problem 3 (Threads)

In Stud.IP you can find a skeleton for a thread implementation using `longjmp` as presented in the lecture.

(a) Implement the missing parts in the functions `schedule` and `init` in `threads.c`.

(b) Install a timer in function `init` and a corresponding signal handler for the signal `SIGALRM` (Example in `timer.c`, see Stud.IP) and call `schedule` there to achieve an automatic switching between the threads. Pay attention to the signal mask.

(c) In the example program for (b), `printf` is called in both `print1` and `print2` but `printf` is not reentrant. Why is the program incorrect and how can `printf` be used correctly?

Note: When copying values of type `jmp_buf`, do not use assignments (such as `jmpbufs[0] = jmpbufs[1]`, this is not guaranteed to work) but use `memcpy`:

```
memcpy(&jmpbufs[0], &jmpbufs[1], sizeof(jmpbufs[0]));
```

You can hand your programs in by uploading them in the folder "Submissions" in Stud.IP; upload source files only. Hand-in is optional.

Due date: Thursday, October 29 at 18:00, discussion on October 30, 2026